Pipeline Case Diagnostics — Conclusion Report

SNNs-auf-GPUs — event-data pipeline verification, regression-task extension, and external novelty review

Summary verdict: All six event-camera datasets in the registry (N-MNIST, N-Caltech101, DAVIS Camera Pose, DVS128 Gesture, DSEC, Eye Tracking) now process correctly end-to-end through the real training pipeline. Seven distinct, previously-latent bugs were found and fixed in the process — all general, dataset-agnostic fixes, not per-dataset patches. A self-contained regression-task framework (4 backends, vector and dense-flow architectures) was built and verified on cached data. Two independent external reviews (Gemini, Perplexity) confirmed the data pipeline's resource-adaptive design is a genuine systems-level assembly not documented elsewhere in the SNN ecosystem, while also identifying the one concrete gap that still needs closing: cross-framework neuron-equivalence verification, which the project's existing comparative claims currently depend on without having.

1 — What was being diagnosed

The starting question was simple: can every dataset registered in `event_data_workflow/dataset_registry.py` actually be loaded and processed by the real pipeline (`NeuromorphicEncoder` → `DataLoader` → a real batch), not just in an isolated test, but the way a real run at the terminal would exercise it? Five of six datasets had never been run through the pipeline before this session; only N-MNIST had a working, cached history.

The project's stated design goal — genuinely dataset-agnostic, not dataset-agnostic by convention — was treated as the actual test criterion: if a dataset requires special-casing to work, the pipeline isn't what it claims to be. Every bug found below was fixed at the mechanism level (in `data_pipeline.py`, `cache_engine.py`, or `system_monitor.py`), not patched around for the one dataset that happened to expose it.

2 — Dataset-by-dataset result

DATASET	KIND	RESULT	WHAT IT NEEDED
N-MNIST	Classification	**Working**	Already functional; used as the control
N-Caltech101	Classification	**Working**	RAM-aware DataLoader worker sizing (§3.1)
DVS128 Gesture	Classification	**Working**	Confirmed the WAF-bypass download URL still functions (real GET, not HEAD)
DAVIS Camera Pose	Regression	**Working**	`ComposedTransform` fix + `FixedToFrame` fix (§3.2, §3.3)
Eye Tracking (3ET)	Regression	**Working**	Same two fixes as DAVIS, plus the float64→float32 dtype fix (§3.6) for the regression build
DSEC	Regression (dense flow)	**Working**	Curated-recording + cache-prewarming fix (§3.4) and batch-size clamp (§3.5)

All six are now confirmed via a real forward pass through the actual `NeuromorphicEncoder` pipeline — not a synthetic smoke test. DSEC and DAVIS are currently configured to their *full* collections (18 optical-flow recordings; 24 Event-Camera-Dataset sequences) per an explicit decision to keep them installable at scale, though only curated single-recording subsets have actually been exercised end-to-end so far — the full-scale runs are a deliberate follow-up, not run in this session, for disk-space reasons (§5).

3 — Bugs found and fixed

Every fix below is general — it changes behavior for every dataset that hits the same condition, not just the one that surfaced it.

3.1 — DataLoader worker count was blind to per-dataset payload size

`SystemResourceMonitor.dataloader_config()` sized `num_workers` from physical core count alone, with no awareness of how large one dataset's batch actually is. For a large-sensor dataset (N-Caltech101, 240×180), enough workers × prefetch batches could exceed Windows' shared-memory commit limit — `RuntimeError: Couldn't open shared file mapping (error 1455)`, a real crash on the very first batch. **Fix:** worker count is now capped by a real measured per-batch byte size against a configurable RAM budget (`resource_policy.worker_ram_fraction`), mirroring the same technique `compute_prefetch_depth()` already used on the GPU/VRAM side.

3.2 — tonic's own `Compose` silently skips `ToFrame` on sparse input

`tonic.transforms.Compose.__call__` breaks out of its transform loop the instant an intermediate result goes empty (`if len(events) == 0: break`). For a sparse window (common in DAVIS/Eye-Tracking's short, mocap-defined windows — some had as few as 8 raw events), `Denoise` can legitimately empty the array, and `ToFrame`'s own correct "return zeros" handling for that case never runs — the raw, now-empty structured event array gets returned instead of a dense frame, crashing

downstream with *"can't convert np.ndarray of type numpy.void."* **Fix:** replaced `tonic.transforms.Compose` with this project's own non-short-circuiting `ComposedTransform` for the Denoise→ToFrame chain.

3.3 — tonic's `ToFrame` swaps H/W on its own empty-input branch

A second, independent tonic bug surfaced once 3.2 was fixed: `ToFrame`'s empty-input zero-fill branch returns shape `(T, C, W, H)`, while its real (non-empty) output is documented and confirmed as `(T, C, H, W)` — swapped axes. Invisible on square sensors (N-MNIST, DVS128 Gesture); breaks `torch.stack` the moment a batch mixes a sparse and a non-sparse sample on a non-square sensor (240×180, 640×480). **Fix:** a small `FixedToFrame` wrapper detects the swapped shape and corrects it, applied uniformly to every dataset's frame transform.

3.4 — DSEC's full recording set causes catastrophic cache thrashing

One DSEC recording alone is ~134 million events (~3GB resident once cached). The full optical-flow "train" split is 18 such recordings, and `WindowedRecordingDataset` only caches one recording at a time — a shuffled `DataLoader` spanning all 18 would evict and reload multiple GB on nearly every batch. **Fix (two parts):** (a) a curated default recording list for quick, safe use; (b) a general `warm_recording_cache()` pass that populates the disk/memory cache in *recording-grouped* order (not the shuffled order `random_split` produces) before the real shuffled `DataLoader` ever starts — so the one expensive pass through each large recording happens once, in order, rather than thrashing. This benefits any current or future multi-recording dataset built on `WindowedRecordingDataset`, not just DSEC.

3.5 — Calibrated batch size can exceed a small dataset's real sample count

`calibrate_batch_size()` sizes batches purely from VRAM fit, with no awareness of how many samples a dataset actually has. DSEC's curated single-recording set has only 41 windows total (32 train after an 80/20 split); a calibrated batch size of 128 combined with `drop_last=True` produced zero batches — `StopIteration` on the very first access. **Fix:** `create_loaders()` now clamps batch size down to the real training-set length whenever the calibrated value exceeds it, so `drop_last=True` always yields at least one batch, for any dataset small enough to hit this.

3.6 — Regression's collate function never downcast tonic's float64 frames

`ToFrame` emits `float64` arrays by default. The classification path uses tonic's own `PadTensors` collate function (which happens not to hit this); the regression path uses this project's own `pad_events_passthrough_target`, which preserved whatever dtype the raw frame arrived in. Under AMP autocast (float16), Conv2d's autocast-halved weights met a float64 input — `RuntimeError: Input type (double) and bias type (Half) should be the same`, on the very first batch of the new regression build. **Fix:** explicit `.float()` cast in the collate function, general to every regression dataset (DAVIS, DSEC, Eye Tracking).

3.7 — `NeuromorphicEncoder` silently re-applies the classification placeholder

Regression datasets carry a placeholder `num_classes=1` in the registry (since "class count" doesn't apply to a continuous target) alongside a real `num_targets` field (7 for pose, 2 for gaze). Setting

`cfg.NUM_CLASSES` to the real target width before building the encoder didn't stick — `NeuromorphicEncoder.load_raw()` re-applies `apply_dataset_shape()` internally using the registry's own placeholder, silently resetting the override. The model's output layer ended up 1-wide instead of the correct width, producing a shape-mismatch warning in the loss computation. **Fix:** `main_regression.py` re-applies the override *after* the encoder is built, immediately before model construction.

4 — New capability: the regression framework

A self-contained regression-task build was added under `frameworks/regression/` (gitignored — experimental sandbox, not part of the tracked classification pipeline) to validate that the pipeline generalizes past classification, not just past "which classification dataset."

COMPONENT	WHAT IT IS
4 vector-regression models	`snn_torch_regression.py`, `snn_norse_regression.py`, `snn_spikingjelly_regression.py`, `snn_sinabs_regression.py` — same Conv-LIF backbone as the classification models, spiking output layer replaced with a non-spiking linear readout, MSE loss, mean-over-T aggregation instead of sum-then-argmax. For DAVIS pose (7-DOF) and Eye Tracking (gaze x,y).
DSEC's dense-flow model	`snn_torch_dense_regression.py` — a genuinely different architecture: spiking Conv encoder + `ConvTranspose2d` decoder, producing a full-resolution per-pixel (u,v) flow field rather than a resized FC output. Masked MSE loss using DSEC's own validity channel; End-Point-Error (the standard optical-flow metric) at test time.
Self-contained trainer/tester	`regression_trainer.py` / `regression_inference.py` (vector case) and `dense_regression_trainer.py` / `dense_regression_inference.py` (DSEC) — kept separate from the classification `SNNTrainer` / `SNNTester` deliberately, since that pipeline's accuracy/argmax/confusion-matrix logic is classification-only.
Entry point	`main_regression.py`, mirroring `learning/main.py`'s structure for the regression-only path.

All of the above were verified with real training + test runs on cached data: Torch, Norse, SpikingJelly, and Sinabs all trained and reported real MSE/RMSE/MAE on Eye Tracking without crashing; the DSEC dense model trained cleanly (masked MSE 147.5 → 142.3 over 2 epochs) and reported a real End-Point-Error of ≈9.0 px across 560,223 valid pixels on test.

5 — Documentation and workspace cleanup

- **Stale documentation corrected across the repo** — the `cfg.TIMESTEPS=25` vs. real `N_TIME_BINS=16` mismatch (a genuine bug: every SynOps/firing-rate figure in prior runs was off by ~1.56×, and the VRAM calibration probe was measuring at the wrong sequence length) was removed entirely; T now derives from the data pipeline's own frame count, one source of truth. STDP references (removed from the codebase but still described as live in several docs) were purged or corrected to past tense. `src/learning/...` paths (stale from a since-removed directory layout) were fixed across README.md, learning/README.md, and several docs/ files.

BindsNET/Spyx — explored and dropped backends — were removed from active discussion throughout, per explicit request, while their dedicated historical records (session log, results table) were preserved.

- `snnbench` **workspace (separate project)** — evaluated for what's worth extracting. Verdict: the equivalence-testing methodology (its single most valuable idea) is directly relevant and not yet adopted here; most of the rest (its own hypothesis-testing machinery) answers snnbench's own research questions, not this project's, and doesn't need porting.
- `tonic inspection` **workspace (separate project)** — its exploratory/visual purpose (understanding what these datasets' events and frames actually look like before committing to sensor sizes and windowing strategies) is fulfilled and reflected in this repo's tested dataset registry. DHP19 and EBSSA were investigated there as candidate regression datasets and explicitly not adopted; Eye Tracking (unrelated to that workspace) was chosen instead.

6 — External validation (Gemini, Perplexity)

Two independent AI reviews were run against a detailed, mechanism-level description of the data pipeline's resource-adaptive design (live-VRAM-probed batch sizing, dataset-length-derived iteration count, adaptive RAM/disk cache tier selection, VRAM-budgeted prefetch depth with explicit double-booking prevention) and separately against the SNN-specific design choices (deriving BPTT length from the data pipeline's own binning parameter; verifying numerical equivalence across four framework backends before comparing them).

QUESTION	CONVERGENT FINDING
Is the data-pipeline design already documented elsewhere?	**No.** Both reviews found the individual mechanisms are standard practice (PyTorch Lightning's batch-size finder, HuggingFace's `auto_find_batch_size`, ordinary `DataLoader` semantics), but found no evidence that this specific combination — applied to SNN training on event-camera data — is documented in BindsNET, Norse, SpikingJelly, Lava, snnTorch, or published SNN benchmarking work.
Is that combination a research contribution, or engineering hygiene?	**Unresolved without a baseline.** Both reviews were explicit: the claims ("dataset-agnostic," "hardware-agnostic," "fast," "memory-managed") need to be restated as measurable hypotheses with a fixed-configuration baseline to compare against (wall-clock, peak memory, crash rate), not asserted as adjectives. This baseline does not yet exist as a run result — the toggles to produce it (`calibrate_batch_size: false`, `calibrate_prefetch_depth: false`) already exist in the config.
Is deriving T from the pipeline's binning parameter SNN-specific and novel?	**SNN-specific: yes. Novel: no.** Both call it a sound consistency rule / interface-correctness improvement — real and worth having, not a new algorithm.
Is cross-framework neuron-equivalence verification necessary here?	**Yes, specifically because this project already makes cross-framework comparative claims.** Both reviews independently flag that without it, the runtime/accuracy comparisons already published in `docs/framework_comparison_report.md` and `docs/results/README.md` are not yet interpretable — this is the same gap the `snnbench` evaluation (§5) identified as the highest-value thing to adopt from that workspace, now confirmed externally.

7 — Open items

1. **Cross-framework equivalence check** — not yet built. Scoped as the smallest useful slice: port `subthreshold_trace` and a trimmed `check_subthreshold` pattern (from the `snnbench` evaluation) into a new small module, run once across all four backends at their currently-configured parameters. Directly closes the validity gap both external reviews flagged.
2. **Baseline-vs-adaptive comparison run** — not yet run. Needed to convert the "novel assembly" verdict into a measured, defensible claim (throughput delta, peak-memory delta, crash rate) rather than a qualitative one.
3. **Full-scale DSEC and DAVIS runs** — configured (18 recordings / 24 sequences) but not yet executed; deferred for disk-space management, to be run one dataset at a time with the install → process → delete protocol.
4. **Classification-dataset stress-test addition** — not completed. N-Cars does not exist in tonic 1.6.0; POKERDVS and ASLDVS both have dead/broken download hosts, confirmed directly (DNS NXDOMAIN for POKERDVS; a corrupted 98KB response for ASLDVS). No further candidate has been identified.

8 — Conclusion

The pipeline's core claim — genuinely dataset-agnostic, not dataset-agnostic in the datasets it happened to already be tested against — held up under direct stress-testing, but only after seven real, previously-undiscovered bugs were found and fixed by actually running every registered dataset through the real pipeline rather than trusting that "it should work." Every fix generalizes past the dataset that exposed it. The regression-task extension confirms the same architecture and resource-management layer holds for a materially different task type, including a dense-prediction case (DSEC) that needed a genuinely different model architecture, not just a resized output layer.

External review independently corroborates that the resource-adaptive design is a real, undocumented systems contribution as an assembly — and independently arrives at the same next step already identified from this project's own prior work: cross-framework neuron equivalence is the one piece of rigor this project's existing comparative claims currently assume rather than prove.